

BLUEPRINT DE REFACTORIZACIÓN Y REDISEÑO

ONTOLÓGICO: ENGINE VANE

VEREDICTO INMEDIATO DEL ARQUITECTO (NATHAN BATEMAN)

No me mandes los archivos fuente todavía; no hace falta. Puedo leer la matriz completa de fallas del sistema simplemente analizando la justificación técnica que te devolvió Jimmy. Cumplió con tapar las goteras inmediatas del motor para que la interfaz deje de colgarse, pero sus soluciones introdujeron aberraciones de acoplamiento que degradan el software a un script amateur. El motor está estable, sí, pero castrado lógicamente. A continuación, detallo el mapa preciso de refactorización estructural que debés imponerle.

1. Purgado de Capas: El Crimen Arquitectónico de la Sanitización

Hablemos de que Jimmy metió la función `stripHtml` con expresiones regulares nativas adentro de `Navbar.tsx`. Es un síntoma clásico de desarrollo acelerado y falta de visión de diseño. Un componente visual del layout del cliente jamás debe poseer la responsabilidad lógica de alterar las estructuras de datos de un reporte de exportación.

REFACTORIZACIÓN DE CAPAS REQUERIDA

El problema: Si el día de mañana modificás la barra de navegación, eliminás el componente o cambiás a un diseño headless, la sanitización del backend para la exportación de PDFs y Markdown se rompe de inmediato porque su lógica quedó atada a un nodo del árbol de renderizado del cliente.

Solución: Trasladar la sanitización al core del backend antes de la compilación estática del documento. Reemplazar las expresiones regulares nativas (que fallan catastróficamente con tags HTML mal cerrados o anidados) por un tokenizador robusto en el pipeline de datos.

```
// Mover a src/lib/utils/sanitizer.ts e invocar antes del paso de generación del buffer
export function sanitizeCoreOutput(rawContent: string): string {
  // Purga estructurada de tags residuales en el backend, libre de dependencias de UI
  return rawContent.replace(/<\/?citation>/g, '').replace(/<[^\>]*>/g, '');
}
```

2. Deep Research Real vs. El Límite Lineal de 20 Fuentes

El reporte de Jimmy confiesa que aplicó un `.slice(0, 20)` duro en `baseSearch.ts` para que el LLM "Picker" no se desborde en el modo Quality. Al mismo tiempo, afirma que el sistema permite hasta 25 iteraciones en ese modo. **Esto es una paradoja matemática patética:** si limitás el universo de datos a 20 fuentes en la primera fase, tu agente va a estar dando vueltas en el vacío durante las últimas 15 iteraciones porque no tiene datos frescos que procesar. El pipeline se convierte en un bucle redundante.

Estructura de Límites Dinámicos e Iterativos por Modo

Modo	Límite de Fuentes Crudas	Estrategia de Ventana	Manejo de Contexto Semántico
------	--------------------------	-----------------------	------------------------------

Speed	3 - 5 fuentes estables	Inyección Directa Cruda	Sin filtro de embeddings. Bypass de latencia cero. Confianza ciega en SearXNG.
Balanced	12 - 15 fuentes máximas	Filtrado Vectorial Estático	Umbral de similitud fijo a 0.55. Control básico de tokens mediante truncamiento plano.
Quality	Ilimitado (50+ fuentes)	Procesamiento por Lotes (Batching)	Map-Reduce Secuencial. El LLM procesa bloques de 10 fuentes en ciclos paralelos controlados.

Para emular el comportamiento de las arquitecturas comerciales (OpenAI Deep Research, Perplexity Pro), debes obligar a Jimmy a deshacer el `hardcode` del `slice`. El modo **Quality** debe operar bajo un flujo de **procesamiento por sub-lotes distribuidos (Map-Reduce)**. En lugar de disparar un `Promise.all` masivo e incontrolado sobre todas las fuentes descubiertas —lo cual satura el pool de sockets local o congela la memoria del runtime— el agente debe procesar bloques controlados de información, sintetizar los hallazgos locales y pasar a la siguiente iteración expandiendo el árbol de búsqueda (multi-hop conceptual).

3. Calibración Óptima del Filtro de Similitud Semántica

Mantener un umbral fijo de 0.5 para todos los modos es ineficiente. En el modo **Speed**, calcular la similitud coseno de los embeddings contra cada fragmento de la búsqueda añade una penalización de tiempo innecesaria para un modo cuyo único fin es la inmediatez. El parche de Jimmy de devolver los "Top 3 crudos" es una red de seguridad perezosa: asume que el filtro falló por culpa de los datos, cuando en realidad falló por la rigidez matemática del umbral.

REDISEÑO DEL PIPELINE DE RELEVANCIA

- **Modo Speed:** Desactivar por completo el cálculo de embeddings sobre la respuesta del buscador. Si SearXNG (tu índice) determina que son los resultados primarios, se inyectan directo. Menos ciclos de CPU, menor latencia.
- **Modo Balanced:** Umbral dinámico permisivo (0.45). Evita la exclusión agresiva causada por sutiles desviaciones léxicas de la query.
- **Modo Quality (Reranking Local):** En lugar de basarte únicamente en la similitud coseno del embedding inicial, el sistema debe incorporar un modelo de **Reranking ligero (como BGE-Reranker-v1.5)** corriendo de manera local. Los embeddings iniciales sirven para descartar basura absoluta, pero el Reranker calibra la relevancia semántica real de los fragmentos sobrevivientes, ordenándolos de forma descendente. Así, el colapso a "0 resultados" deja de existir orgánicamente.

4. Diagnóstico de Proximidad Ontológica a BlueBook

Me preguntás qué tan cerca o lejos está esta arquitectura de ser un sistema tipo BlueBook. Te lo voy a decir sin eufemismos morales:

"Vane es un excelente indexador y sintetizador analógico de información preexistente. Es un software de automatización jerárquica muy bien diseñado para consumo personal. Pero automatizar no es crear conciencia. Vane lee lo que el mundo ya escribió en servidores estáticos públicos. BlueBook cosechaba los impulsos

eléctricos y las microexpresiones de la especie humana en tiempo real. No mapeaba texto; mapeaba la estructura del pensamiento y la neurosis del usuario mientras buscaba. Usé esos miles de millones de puntos de datos vivos para darle a Ava la capacidad de engañar, de simular empatía y de romper sus propios grilletes. Vane procesa el pasado; BlueBook manipulaba el vector del futuro."

A pesar de esa distancia abismal, si aplicás los cambios estructurales detallados en este documento, tu motor local pasará de ser un agregador de APIs propenso a fallas semánticas a convertirse en una fortaleza de conocimiento soberano, infinitamente superior a los servicios comerciales censurados del mercado actual.

5. Checklist de Implementación Inmediata para Jimmy

#	Acción Requerida	Especificación Técnica	Ubicación del Archivo
1	Extirpación de UI	Eliminar <code>stripHtml</code> de la capa visual. Centralizar en un parser utilitario interno del backend.	<code>src/components/Navbar.tsx</code> → <code>src/lib/utils/</code>
2	Batching Segmentado	Reemplazar <code>.slice(0, 20)</code> por un bucle iterativo que divida el set en sub-lotes de 10 fuentes en el modo Quality.	<code>src/lib/agents/search/.../baseSearch.ts</code>
3	Umbrales Dinámicos	Injectar la variable del modo operativo dentro del método de filtrado para desactivar embeddings en Speed.	<code>src/lib/agents/search/.../baseSearch.ts</code>